

College NSS Application

UI/UX Implementation Specification

Product UI/UX and Frontend Implementation Specification

Frontend: React + TypeScript + Vite

Purpose: Define the complete user interface, navigation model, role-aware experiences, screens, components, states, workflows, and responsive behavior.

1. Product UI Vision

The application is a complete college NSS application used to operate, coordinate, participate in, document, and report NSS activities. The UI must not be designed as an “admin portal.” It should present a coherent application in which each user receives the tools appropriate to their NSS responsibilities.

- Volunteers
- Student Leaders
- Programme Officers
- Faculty Coordinators
- Administrators

Core principle: One college NSS application, with role-aware capabilities.

2. UX Principles

Role-aware, not role-dominated

Roles control navigation visibility, available actions, editable fields, approval capabilities, reporting access, and data scope. Roles should not determine completely different applications.

Workflow-first design

The UI should represent real NSS workflows rather than database entities alone. A programme should naturally connect to registration, attendance, service hours, reports, and recognition.

Information hierarchy

Every screen should clearly communicate where the user is, what is happening, what requires attention, what can be done, and what has already happened.

Progressive disclosure

Do not expose every possible action immediately. Start with essential information and reveal operational details in detail views.

Responsive by default

The application must work across desktop, laptop, tablet, and mobile. Desktop is primary for management-heavy workflows; mobile is especially important for volunteers, attendance, QR workflows, announcements, and quick actions.

3. Global Application Shell

NSS Logo / College Name		Search	Alerts	User
-----		-----		
Navigation		Page Content		
Overview				
Volunteers				
Units				
Programmes				
Attendance				
Service Hours				
Reports				
Documents				
...				

Desktop

- Controlled sidebar
- Top application bar
- Content container
- Breadcrumbs where useful
- Contextual page actions

Mobile

- Compact header
- Menu drawer
- Bottom navigation for high-frequency actions where useful
- Full-width content
- Bottom sheets for lightweight actions

4. Branding

The UI should look like a college institutional application rather than a generic SaaS dashboard.

- Clean
- Professional
- Accessible
- Academic/institutional
- Modern without being flashy
- Information-dense where necessary
- Calm visual hierarchy

Use college/NSS identity consistently across login, application header, dashboards, reports, certificates, event pages, and empty states.

5. Global Navigation

Overview

People

- Volunteers
- Student Leaders

Organisation

- NSS Units

Programmes

- Events
- Activities
- Camps

Participation

- Registrations
- Attendance
- Service Hours

Recognition

- Achievements
- Certificates

Communication

- Announcements
- Notifications

Documents

- Activity Reports
- Documents

Reports & Analytics

Administration

Not every role sees every navigation item. Navigation is generated from the user's permissions and responsibilities.

6. Overview

The Overview screen is the application's operational starting point. It should answer: What is happening in NSS right now, and what requires attention?

Common sections

- Summary: active volunteers, NSS units, upcoming programmes, service hours, recent activities
- Upcoming: events, registration state, date/time, venue, unit

- Attention Required: pending attendance verification, service-hour corrections, report deadlines, unpublished programmes, registrations nearing capacity
- Recent Activity: enrollment, event registration, attendance completion, service-hour updates, certificates

7. Volunteer Experience

7.1 Volunteer list

- Search
- Filters
- Pagination
- Sorting
- Status
- Department
- NSS unit
- Academic year
- College ID
- Participation status

Name | College ID | Department | Unit | Status | Hours | Actions

7.2 Volunteer profile

Volunteer

Profile

Name / College ID / Department / Year / Contact / Status

Current NSS Unit

Unit / Role / Joined Date

Participation

Events attended / Service hours / Attendance rate

Achievements

Certificates

Membership History

Recent Activity

7.3 My NSS

- Upcoming Events
- My Registrations
- Attendance
- Service Hours
- Achievements
- Certificates
- Announcements
- Profile

8. NSS Units

Unit list

- Unit number
- Unit officer
- Member count
- Active programmes
- Service hours
- Status

Unit detail

Unit 01

Overview

Officer / Member Count / Active Programmes / Service Hours

Tabs:

Members

Programmes

Attendance

Service Hours

Achievements

Reports

Documents

9. Programmes & Events

Event catalogue

- Event title
- Event type
- Date
- Time
- Venue
- NSS unit
- Status
- Available seats
- Registration state

Community Cleanliness Drive

24 September 2026

08:00 - 12:00

NSS Unit 03

College Campus

OPEN

32 / 50 registered

[View Details]

Event detail

Event Title / Status / Date / Time / Venue / Unit

Description

Registration

Registered / Capacity / Remaining / Registration Window

Attendance

Service Hours

Participants

Activity Report

Documents

Timeline

Event lifecycle

DRAFT → PUBLISHED → OPEN → CLOSED → COMPLETED

→ CANCELLED (terminal)

The frontend must respect the backend lifecycle and authorization. Displaying an action must never be treated as authorization.

10. Event Creation

1. Basic information: title, event type, description, NSS unit
2. Schedule: start/end, registration opening/closing
3. Location: venue
4. Capacity: maximum participants
5. Review: summary before submission

Event Summary

Title

Unit

Date

Time

Venue

Capacity

Registration Window

[Save Draft]

11. Registration UI

Registration

32 / 50 registered

Registration closes: 23 Sep, 6:00 PM

[Register]

After registration, show confirmation, registration ID, registration time, attendance instructions, and an option to cancel when allowed.

Explicit states should be shown for closed registration and reached capacity.

12. Attendance

Attendance dashboard

- Upcoming attendance sessions
- Active sessions
- Completed sessions
- Pending verification
- Attendance statistics

Attendance session

Event

Date

Venue

Attendance Session

Status: ACTIVE

Registered: 50

Present: 37

Pending: 13

[Start Attendance]

[Close Session]

QR attendance

QR attendance should be mobile-first. The server remains authoritative for session validity and attendance recording.

Attendance roster

Show volunteer, registration state, attendance state, timestamp, and correction status, with filters for present, absent, pending, and corrected.

13. Attendance Corrections

Attendance Correction

Volunteer: Rahul Kumar

Event: Community Cleanliness Drive

Current: Absent

New: Present

Reason:

[_____]

[Submit Correction]

Corrections should visibly communicate that a record is being changed and should expose approval/audit context where appropriate.

14. Service Hours

Service hours should be presented as formal verified records, not merely a number.

My Service Hours

Total Verified Hours: 84.5

This Academic Year: 42.0

Recent Entries

Event	Hours	Status
Cleanliness Drive	4.0	Verified
Blood Donation Camp	5.0	Verified
Awareness Programme	3.0	Pending

Entry details should show event, volunteer, date, hours, source, verification state, creator, and correction history where permitted.

15. Achievements

- Best Volunteer
- Outstanding Participation
- Community Service
- Leadership
- Special Contribution

Keep recognition institutional rather than overly gamified.

16. Certificates

My Certificates

Community Service Participation

2026

Issued: 24 September 2026

[View] [Download]

Certificates should be traceable to the underlying activity, achievement, or participation record.

17. Announcements

- Title
- Published date

- Author/office
- Audience
- Priority
- Read state

Announcements are official NSS communication and may include attachments and related programmes.

18. Notifications

- Registration confirmation
- Registration cancellation
- Event reminder
- Attendance confirmation
- Service-hour verification
- Certificate availability
- Announcement
- Report deadline

Notifications can be unread, read, or actionable.

19. Documents

- Activity reports
- Event documents
- Supporting documents
- Certificates
- Institutional documents

Display name, category, related event/unit, uploaded date, uploader, and status. Do not expose raw storage details.

20. Activity Reports

Activity Report

Programme

Community Cleanliness Drive

Date

24 September 2026

Unit

NSS Unit 03

Participants

48

Service Hours

Summary
Outcomes
Photos / Documents

Report Status
DRAFT / SUBMITTED / VERIFIED

Reports should connect to their underlying event and participation data wherever possible.

21. Reports & Analytics

Report categories

- Volunteers: active volunteers, volunteers by unit, participation, service hours
- Programmes: conducted, upcoming, completed, cancelled
- Attendance: rates, participation, exceptions, corrections
- Service Hours: total verified hours, by unit, by volunteer, trends
- Institutional reporting: academic-year, unit, activity, and participation summaries

Filters

- Academic year
- Date range
- NSS unit
- Event type
- Department
- Volunteer
- Status

Filters should persist in the URL where practical.

Export

- CSV
- Spreadsheet-compatible exports
- PDF reports where appropriate

Exports must respect backend authorization.

22. Administration

Administration is a specific area, not the identity of the application.

- User administration
- Roles
- Permissions
- System configuration
- Audit logs

- Controlled data management

23. Audit Logs

Timestamp
Actor
Action
Resource
Resource ID
Result

Audit information is for traceability. Sensitive technical information should not be exposed unnecessarily.

24. Search

Global search should support relevant NSS objects such as volunteers, events, units, announcements, documents, and certificates. Search results must respect authorization.

25. Forms

- Labels above fields
- Clear required indicators
- Inline validation
- Server validation
- Helpful error messages
- Disabled state during submission
- Success confirmation
- Preservation of entered data where possible

Avoid vague errors such as “Something went wrong.” Prefer specific domain messages such as “Registration is closed for this event.”

26. Tables

- Pagination
- Sorting
- Filtering
- Responsive behavior
- Empty states
- Loading states
- Row actions
- Keyboard accessibility

On mobile, use cards for complex records or carefully designed horizontal tables only when necessary.

27. Cards

- Event discovery
- Unit summaries
- Certificates
- Achievements
- Dashboard summaries

Use tables for operational records when comparison across rows is more useful than visual cards.

28. Modals and Drawers

- Modals: short forms, confirmations, quick actions
- Drawers: detail previews and secondary workflows
- Full pages: complex forms, event management, profiles, reports, attendance, service-hour records

29. Loading States

- Every data-driven screen needs a deliberate loading state
- Use skeletons for larger areas
- Never display a blank screen while waiting for an API

30. Empty States

Empty states should explain what happened and what to do next.

No upcoming programmes

Programmes created for your unit will appear here.

31. Error States

API Error	UI Message
AUTHENTICATION_REQUIRED	Please sign in again.
FORBIDDEN	You do not have permission to perform this action.
RESOURCE_NOT_FOUND	This NSS record could not be found.
VALIDATION_ERROR	Please correct the highlighted fields.
CONFLICT	This record conflicts with an existing record.
DUPLICATE_REGISTRATION	You are already registered for this event.
REGISTRATION_CLOSED	Registration for this event is closed.
EVENT_CAPACITY_REACHED	This event has reached its participant capacity.
ATTENDANCE_SESSION_EXPIRED	The attendance session has expired.
DUPLICATE_ATTENDANCE	Your attendance has already been recorded.
RATE_LIMITED	Too many requests. Please try again shortly.

32. Permission-Aware UI

Frontend permissions improve experience but are not security controls. Buttons and routes may be hidden based on permissions, but every API operation must be authorized by the backend.

```
if user.canCreateEvent
  show Create Event
```

```
if user.canManageAttendance
  show Attendance Management
```

```
if user.canViewReports
  show Reports
```

33. Role Experience Matrix

Capability	Volunteer	Student Leader	Programme Officer	Faculty Coordinator	Administrator
View own profile	✓	✓	✓	✓	✓
Register for event	✓	✓	✓	✓	✓
View own attendance	✓	✓	✓	✓	✓
View own service hours	✓	✓	✓	✓	✓
Manage unit operations	—	Scoped	Scoped	Broader	✓
Create programmes	—	Permission-based	✓	✓	✓
Manage attendance	—	Scoped	✓	✓	✓
Verify service hours	—	Permission-based	✓	✓	✓
View reports	Personal	Unit	Operational	Institutional	✓
Manage users/roles	—	—	—	As authorized	✓
Audit logs	—	—	Limited/appropriate	Appropriate	✓

34. Dashboard Personalization

Volunteer

- My NSS
- Upcoming Events
- My Registrations
- My Attendance
- My Service Hours
- Achievements
- Announcements

Student Leader

- Unit Overview
- Upcoming Programmes
- Registrations
- Attendance
- Unit Volunteers
- Service Hours
- Pending Actions

Programme Officer

- Programme Overview
- Upcoming Programmes
- Attendance Verification
- Service-Hour Records
- Activity Reports
- Unit Participation
- Pending Actions

Faculty Coordinator

- College NSS Overview
- Units
- Volunteers
- Programmes
- Participation
- Service Hours
- Reports
- Institutional Activity

Administrator

- Application Administration
- Users
- Roles
- Permissions
- System Status
- Audit
- Configuration

35. Mobile UX

Volunteer navigation

Home | Events | Attendance | Hours | Profile

Operational navigation

Overview | Programmes | Attendance | People | More

- Large touch targets
- Bottom sheets
- Compact filters
- Sticky contextual actions
- Camera/QR workflows
- Simplified tables

36. Accessibility

Target WCAG 2.1 AA principles.

- Keyboard navigation
- Visible focus
- Semantic HTML
- Proper form labels
- Sufficient contrast
- Accessible modal behavior
- Screen-reader-friendly status messages
- No color-only meaning
- Accessible tables
- Accessible error messages

37. Design System

Layout

- AppShell
- Sidebar
- Topbar
- MobileNav
- PageHeader
- Breadcrumbs
- ContentContainer

Data

- DataTable
- Pagination
- SearchInput
- FilterBar
- EmptyState
- LoadingState
- ErrorState

Forms

- FormField

- Select
- DateTimePicker
- TextArea
- FormActions

Feedback

- Alert
- Toast
- ConfirmationDialog
- StatusBadge
- ProgressBar

NSS-specific

- EventCard
- EventStatusBadge
- EventCapacity
- VolunteerCard
- UnitCard
- AttendanceStatus
- ServiceHourSummary
- CertificateCard
- AchievementCard
- ProgrammeTimeline

38. URL Architecture

```

/
├── dashboard
├── volunteers
│   ├── /volunteers
│   └── /volunteers/:id
├── units
│   ├── /units
│   └── /units/:id
├── events
│   ├── /events
│   ├── /events/new
│   └── /events/:id
├── attendance
│   ├── /attendance
│   └── /attendance/:id
├── service-hours
│   ├── /service-hours
│   └── /service-hours/:id
├── achievements
├── certificates
├── announcements
├── documents
└── reports

```

```
└─ administration
   └─ users
   └─ roles
   └─ permissions
   └─ audit
```

39. Frontend State Architecture

Server state

- Volunteers
- Units
- Events
- Registrations
- Attendance
- Service hours

Session state

- Authenticated user
- JWT/session state
- Permissions
- Role information

UI state

- Modal state
- Filters
- Table view
- Temporary form state
- Sidebar state

40. API Integration

All API communication should pass through the typed API client.

```
src/
├─ api/
│   ├── client.ts
│   ├── auth.ts
│   ├── volunteers.ts
│   ├── units.ts
│   ├── events.ts
│   ├── attendance.ts
│   └── serviceHours.ts
├─ components/
├─ features/
│   ├── auth/
│   ├── volunteers/
│   ├── units/
│   └── events/
```

```
| |— attendance/
| |— service-hours/
| |— reports/
| |— certificates/
|— layouts/
|— pages/
|— routes/
|— types/
|— styles/
```

41. Event Frontend Example

```
features/events/
|— components/
|   |— EventCard.tsx
|   |— EventStatusBadge.tsx
|   |— EventCapacity.tsx
|   |— EventFilters.tsx
|   |— RegistrationButton.tsx
|   |— EventTimeline.tsx
|— pages/
|   |— EventsPage.tsx
|   |— EventDetailPage.tsx
|   |— EventCreatePage.tsx
|— api.ts
|— types.ts
|— hooks.ts
```

42. UX for Important NSS Workflows

Volunteer onboarding

Create Volunteer → Profile Information → College Information → NSS Unit → Membership → Active Volunteer

Programme lifecycle

Draft → Published → Registration Open → Registration Closed → Attendance → Completed → Report → Service Hours

Volunteer participation

Discover Event → View Details → Register → Attend → Attendance Verified → Service Hours Recorded → Achievement / Certificate

43. Dashboard Metrics

Metrics should always have context.

84.5

Verified Service Hours

This Academic Year

Avoid displaying unexplained numbers. Each metric should identify its population, period, and meaning where applicable.

44. Data Visualization

- Service hours over time
- Participation by unit
- Events by type
- Volunteer participation
- Attendance trends

Use charts only where they improve understanding. Simple data should remain simple.

45. Notifications and Confirmation

Important operations should provide immediate feedback. Destructive or sensitive actions should explicitly communicate consequences and require appropriate confirmation.

46. Security UX

- Never display passwords or password hashes
- Never expose JWT secrets or database credentials
- Never expose stack traces
- Do not leak internal infrastructure details
- Unauthorized screens should not reveal unnecessary resource information

47. Performance

- Pagination for large lists
- Server-side filtering
- Debounced search
- Lazy loading for large modules
- Avoid loading all volunteers/events at once
- Cache appropriate server state
- Avoid unnecessary API requests
- Use optimistic updates only where safe

48. Responsive Breakpoints

Mobile: < 640px

Tablet: 640px – 1024px

Desktop: > 1024px

Exact CSS values can be refined during implementation, but breakpoints should remain consistent across the design system.

49. Visual Density

Management-heavy screens may contain tables, filters, metrics, and actions. Volunteer screens should generally be simpler. Not every page should have identical visual density.

50. Design Consistency Rules

- Typography scale
- Spacing system
- Button hierarchy
- Form styling
- Status language
- Border radius
- Card behavior
- Table patterns
- Error handling
- Loading behavior
- Responsive conventions

A new module should feel like it belongs to the same application without requiring a new design language.

51. Implementation Priorities

6. Application shell: authentication, navigation, responsive shell, dashboard, session state, common components
7. Core NSS data: volunteers, profiles, NSS units, unit details
8. Programmes: event catalogue, detail, creation, lifecycle, registration
9. Participation: attendance, QR attendance, rosters, corrections, service hours
10. Communication: announcements and notifications
11. Recognition: achievements and certificates
12. Documentation: documents and activity reports
13. Reporting: dashboards, analytics, exports
14. Administration: users, roles, permissions, audit, configuration

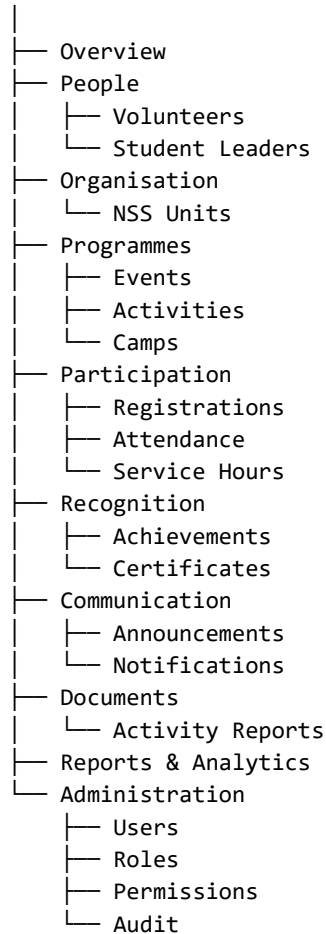
52. Definition of UI Completion

- Loading state
- Empty state
- Error state
- Permission-aware actions
- Responsive layout
- Validation
- Success feedback
- Server error handling
- Pagination where needed

- Accessibility
- Deep-linkable routes
- API integration
- Confirmation for destructive operations

53. Final UI Architecture

COLLEGE NSS APPLICATION



54. Product-Level UI Goal

The final UI should allow someone from the college to open the application and immediately understand what is happening in NSS, which programmes are upcoming, who is participating, which units are active, who attended, how many service hours were earned, which activities have been completed, what reports exist, what achievements and certificates have been issued, and what actions require attention.

At the same time, a volunteer should not be overwhelmed by institutional management functions, and a Programme Officer should not be forced through a volunteer-oriented interface to perform operational work.

The result should be a single, coherent, role-aware college NSS application with a consistent information architecture and a clear relationship between programmes, participation, attendance, service hours, recognition, documentation, and reporting.

College NSS Application — UI/UX Implementation Specification